



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 08

Team No.: 11

Project Title: Linux Inter-Process Communication Using Pipes and FIFOs

Team Members

Roll Number	Student Name	Signature
2520030016	Hamsini Cherukuri	
2520030020	Varnika. V	
2520030352	Minajja Reddy	
2520030580	Likhitha. B	

1. Abstract

Linux Inter-Process Communication Using Pipes and FIFOs** is an Operating Systems project that demonstrates how processes in a Linux environment communicate and exchange data using **Inter-Process Communication (IPC)** mechanisms. The project focuses on two important IPC techniques: **Pipes** and **FIFOs (Named Pipes)**. Pipes provide a simple way for related processes, such as parent and child processes, to transfer data through a unidirectional communication channel. FIFOs extend this concept by allowing unrelated processes to communicate using a named communication channel available in the file system.

The project implements practical examples of creating, writing, reading, and closing pipes and FIFOs using **Linux system calls and C programming**. It demonstrates how data flows between processes, how synchronization is achieved during communication, and how processes coordinate their execution. The project also highlights the differences between anonymous pipes and named FIFOs in terms of process relationship, accessibility, and usage. Through these implementations, the project provides a practical understanding of Linux IPC concepts and demonstrates how efficient process communication can be achieved in a multi-process operating system environment.

2. Problem Statement

In a Linux operating system, multiple processes often need to communicate and exchange data while executing concurrently. Without proper Inter-Process Communication (IPC) mechanisms, it becomes difficult for processes to share information reliably and coordinate their execution. Therefore, there is a need to understand and implement efficient IPC techniques that allow processes to communicate in a controlled and synchronized manner.

This project addresses the problem of **process communication in Linux** by implementing two fundamental IPC mechanisms: **Pipes and FIFOs (Named Pipes)**. The project demonstrates how related processes can communicate using anonymous pipes and how unrelated processes can exchange data using named FIFOs. It also focuses on the proper creation, writing, reading, synchronization, and closing of these communication channels using Linux system calls and C programming.

The main objective is to provide a practical understanding of how Linux processes communicate, how data is transferred between processes, and how Pipes and FIFOs differ in terms of accessibility, process relationships, and usage.

3. Objectives

1. To understand the concept of **Inter-Process Communication (IPC)** in Linux operating systems.
2. To implement **anonymous pipes** for communication between related processes such as parent and child processes.
3. To implement **FIFOs (Named Pipes)** for communication between unrelated processes.
4. To demonstrate the **creation, writing, reading, and closing** of Pipes and FIFOs using C programming and Linux system calls.
5. To understand how **data is transferred between processes** through IPC mechanisms.
6. To study the role of **process synchronization and coordination** during inter-process communication.
7. To compare **Pipes and FIFOs** based on process relationship, accessibility, communication method, and practical usage.
8. To gain practical knowledge of **Linux process communication** through hands-on implementation.

4. Proposed Methodology

1. **Study of IPC Concepts**
Understand the fundamentals of Inter-Process Communication in Linux and study the working principles of Pipes and FIFOs.
2. **Environment Setup**
Set up a Linux/Ubuntu environment with the GCC compiler to develop and execute C programs.
3. **Implementation of Anonymous Pipes**
Create an anonymous pipe using the `pipe()` system call and use `fork()` to establish communication between a parent and child process.
4. **Data Writing and Reading**
Implement programs where one process writes data into the pipe and another process reads the data from it using appropriate system calls.
5. **Implementation of FIFOs**
Create a Named Pipe using the `mkfifo()` system call to enable communication between unrelated processes.
6. **Process Synchronization**
Coordinate the execution of communicating processes and ensure proper opening, reading, writing, and closing of the communication channels.
7. **Testing and Validation**
Execute the programs with different inputs and verify that the transmitted data is received correctly by the communicating process.

8. Comparison of Pipes and FIFOs

Compare both IPC mechanisms based on process relationship, communication method, accessibility, and practical applications.

9. Result Analysis

Observe the execution results and confirm that Pipes and FIFOs provide reliable communication between Linux processes.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
pipe()	Creates an anonymous pipe for communication between related processes
fork()	Creates a child process to demonstrate communication between parent and child.
read()	Reads data from the pipe or FIFO.
write()	Writes data into the pipe or FIFO.
mkfifo()	Creates a named FIFO for communication between unrelated processes.
open() / close()	Opens and closes the communication channel used by the processes.
Process Synchronization	Coordinates the execution of processes during data transfer.
Inter-Process Communication (IPC)	Enables processes to exchange data and communicate with each other.

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520030016	Hamsini Cherukuri	Implemented Pipes using pipe() and fork() for parent-child process communication.
2520030020	Varnika. V	Implemented FIFOs using mkfifo() for communication between unrelated processes.
2520030352	Minajja Reddy	Performed testing, process synchronization, error handling, and debugging of the IPC programs.
2520030580	Likhitha. B	Analyzed Pipes and FIFOs, prepared results, screenshots, comparison, and project documentation.

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating system environment for developing and executing IPC programs.
C / C++	Used to implement Pipes, FIFOs, and process

	communication programs.
GCC Compiler	Compiles the C/C++ source code into executable programs.
Linux System Calls	Used for IPC operations such as pipe(), fork(), mkfifo(), read(), write(), open(), and close().
Terminal / Command Line	Used to compile, execute, test, and monitor the IPC programs.

8. Expected Outcome

The project is expected to successfully demonstrate Inter-Process Communication in a Linux environment using Pipes and FIFOs. The implementation will show how related processes can communicate through anonymous pipes and how unrelated processes can exchange data using named FIFOs. The project is expected to demonstrate reliable data transmission, proper process synchronization, and the use of Linux system calls such as pipe(), fork(), mkfifo(), read(), and write(). The results will also provide a clear comparison between Pipes and FIFOs based on their working, process relationships, accessibility, and applications. Overall, the project is expected to provide practical knowledge of Linux IPC and multi-process communication using C programming.

9. GitHub Repository

GitHub Repository Link: https://github.com/varn1k4/DONUT_OSSP

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Muniraju Vadlamudi

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____